GRAPHES BASIQUES

Table des matières

1. L'importance des graphiques
2. Le jeu de données `mtcars`
3. Diagrammes à barres
4. Histogrammes
5. Diagrammes à secteurs

L'importance des graphiques

La visualisation de données est la présentation de données avec des graphiques. C'est un moyen de résumer vos conclusions et de les afficher sous une forme qui facilite l'interprétation et peut aider à identifier des modèles ou des tendances. Avoir de superbes visualisations de données rendra votre travail plus intéressant et plus clair. Dans ce cours, vous apprendrez à utiliser la bibliothèque ggplot2 pour créer de superbes graphes et diagrammes, en personnalisant leur apparence et leur convivialité à votre guise.

La différence entre les bibliothèques R

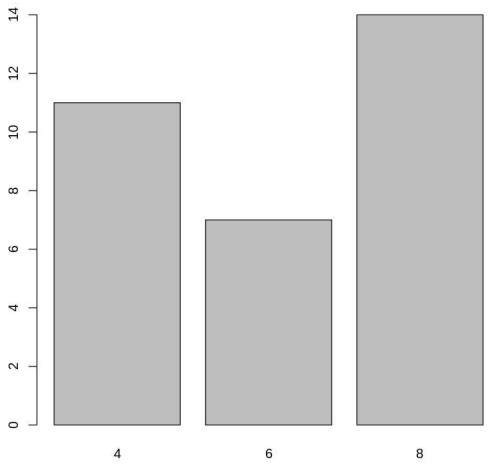
Les différences entre la bibliothèque de base plot(), fournie avec R, et ggplot2 sont nombreuses. Ggplot2 a été créé pour répondre aux demandes de conception et était basé sur le livre "The Grammar of Graphics", un livre qui décrit les bases du traçage de données.

Regardons les différences entre eux à l'aide d'un exemple simple.

plot

La bibliothèque `plot` est la bibliothèque R par défaut pour tracer des graphiques. C'est très simpliste à la fois dans la syntaxe et l'esthétique. Pour l'utiliser pour créer un bar plot, vous utilisez la fonction `barplot`, comme ceci :

```
count <- table(mtcars$cyl)
barplot(count)
```



ggplot2

`ggplot2`, comme mentionné ci-dessus, est une bibliothèque spécialisée conçue pour créer des visualisations de données visuellement agréables. Avant de pouvoir utiliser `ggplot2`, nous devons l'importer dans l'environnement R. La cellule de code ci-dessous vérifiera si votre

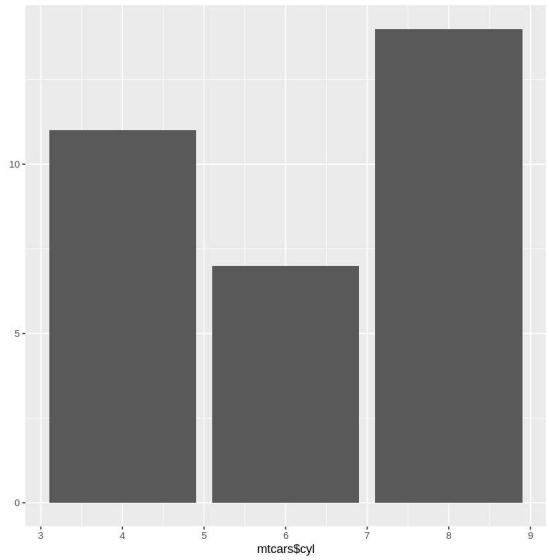
système a déjà ggplot, afin de ne pas exécuter `install.packages` sans raison. Ensuite, en utilisant la fonction `library`, nous pouvons alors importer `ggplot2`.

```
if("ggplot2" %in% rownames(installed.packages()) == FALSE) {install.packages("ggplot2")}
library(ggplot2)
```

Maintenant, traçons notre graphique. Pour tracer un simple graphique à barres en utilisant le `qplot` de `ggplot2`, nous utilisons ceci :

```
qplot(mtcars$cyl, geom = "bar")
```

```
Warning message:
“`qplot()` was deprecated in ggplot2 3.4.0.”
```



Ne vous inquiétez pas - nous allons revenir en arrière et apprendre ces méthodes de traçage et d'autres au cours de nos leçons.

Comme vous pouvez le voir, `ggplot2` nous offre un graphique plus joli, mais a une syntaxe légèrement plus complexe que la bibliothèque `plot` par défaut.

ggplot2 a deux fonctions principales, `qplot()` et `ggplot()`.

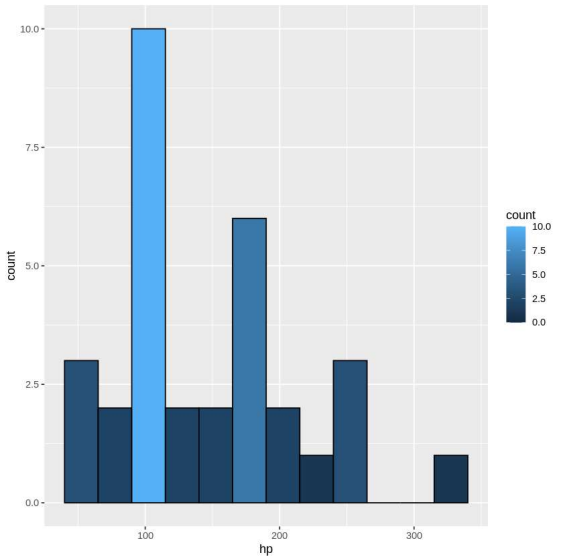
- `qplot()` offre une syntaxe plus simple similaire à la fonction `plot` par défaut, mais sa personnalisation est limitée.
- `ggplot()` est la fonction à part entière. Il a beaucoup plus de personnalisations possibles, mais a une syntaxe plus compliquée que `qplot()`.

Dans ce cours, nous commencerons à utiliser `qplot()` puis passerons à `ggplot()` au fur et à mesure que vous avancez.

Le graphique suivant est une démonstration de ce qu'il est possible de faire avec `ggplot()`. À la fin de ce cours, vous serez capable de créer des graphiques comme celui-ci ci-dessous et des graphiques encore plus complexes.

```
ggplot(data= mtcars, aes(hp))  +
  geom_histogram(breaks=seq(40, 350, by = 25),
                 colour = I("black"),
                 aes(fill=..count..))
```

```
Warning message:
“The dot-dot notation (`..count..`) was deprecated in ggplot2 3.4.0.
i Please use `after_stat(count)` instead.”
```



Données qualitatives vs quantitatives

Une chose à toujours garder à l'esprit est le type de données pour lesquelles vous essayez de créer des graphiques. En général, nous classons les données en deux grands groupes : les données qualitatives (également appelées données catégorielles) et les données quantitatives (également appelées données numériques).

Ce que nous appelons données **qualitatives ou catégorielles** sont des données qui font référence, comme le montre leur nom, à des catégories. Cela peut inclure des données pour les questions « Oui ou Non », il peut s'agir du nom d'un lieu, de la saveur de crème glacée préférée d'une personne ou d'autre chose. C'est quelque chose de très différent des données **quantitatives ou numériques**. Les données quantitatives sont des données qui sont, tout simplement, des nombres. Il s'agit normalement d'une mesure quelconque et peut être manipulé à l'aide de calculs simples.

Les méthodes de traçage et les meilleurs types de graphiques pour chaque type de données sont différents - choisir le meilleur vous aidera grandement à créer des graphiques visuellement agréables.

Le jeu de données mtcars

« mtcars » est un ensemble de données intégré qui contient des données du magazine américain Motor Trend de 1974, et contient la consommation de carburant et 10 autres aspects des performances automobiles pour 32 automobiles de 1973 à 1974.

Comme il est déjà inclus avec R, il n'est pas nécessaire d'importer `mtcars`. Vérifions sa structure - nous pouvons le faire en l'appelant par son nom, comme ceci :

```
mtcars
```

Quels types d'informations pouvons-nous tirer de ces données ? Nous avons le kilométrage des voitures par gallon d'essence, le nombre de cylindres et d'autres attributs.

S'il y a une colonne dont vous ne savez pas ce qu'elle représente, vous pouvez utiliser la fonction `?` pour voir le fichier d'aide pour cet ensemble de données, comme ceci :

```
?mtcars
```

Notez que cela n'est possible que parce que `mtcars` est un jeu de données intégré dans R. Presque aucun ensemble de données n'aura de fichiers d'aide. Certains ensembles de données que vous trouverez sur Internet auront un fichier readme qui décrira ce que chaque colonne représente. Certains incluront le nom de chaque colonne de l'ensemble de données. Bien qu'il s'agisse de bonnes pratiques, vous pouvez trouver des jeux de données avec des noms peu descriptifs pour ses colonnes et des jeux de données sans aucun nom.

Comme vous le savez peut-être déjà, nous n'avons pas besoin d'afficher l'ensemble de données dans son intégralité pour voir comment les données sont structurées. Nous pouvons utiliser la fonction `head` pour voir les 6 premières lignes de nos données.

```
head(mtcars)
```

A data.frame: 6 × 11

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	car
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	

Et nous pouvons utiliser `tail` pour voir les 6 dernières lignes de nos données.

```
tail(mtcars)
```

A data.frame: 6 × 11

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.7	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.9	1	1	5	2
Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.5	0	1	5	4
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.5	0	1	5	6

Analysons plus avant nos données... Nous pouvons obtenir un résumé rapide de chaque colonne en utilisant la fonction « résumé » :

```
summary(mtcars)
```

```
      mpg             cyl             disp             hp       
 Min.   :10.40   Min.   :4.000   Min.   : 71.1   Min.   : 52.0  
 1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5  
 Median :19.20   Median :6.000   Median :196.3   Median :123.0  
 Mean   :20.09   Mean   :6.188   Mean   :230.7   Mean   :146.7  
 3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0  
 Max.   :33.90   Max.   :8.000   Max.   :472.0   Max.   :335.0  
      drat             wt             qsec             vs        
 Min.   :2.760   Min.   :1.513   Min.   :14.50   Min.   :0.0000  
 1st Qu.:3.080   1st Qu.:2.581   1st Qu.:16.89   1st Qu.:0.0000  
 Median :3.695   Median :3.325   Median :17.71   Median :0.0000  
 Mean   :3.597   Mean   :3.217   Mean   :17.85   Mean   :0.4375  
 3rd Qu.:3.920   3rd Qu.:3.610   3rd Qu.:18.90   3rd Qu.:1.0000  
 Max.   :4.930   Max.   :5.424   Max.   :22.90   Max.   :1.0000  
       am              gear            carb      
 Min.   :0.0000   Min.   :3.000   Min.   :1.000  
 1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:2.000  
 Median :0.0000   Median :4.000   Median :2.000  
 Mean   :0.4062   Mean   :3.688   Mean   :2.812  
 3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:4.000  
 Max.   :1.0000   Max.   :5.000   Max.   :8.000  
```

Nous pouvons obtenir la moyenne pour n'importe quelle colonne en utilisant `mean(datasetname$columnname)`, comme ceci :

```
mean(mtcars$cyl)
```

```
6.1875
```

Créer des diagrammes à barres

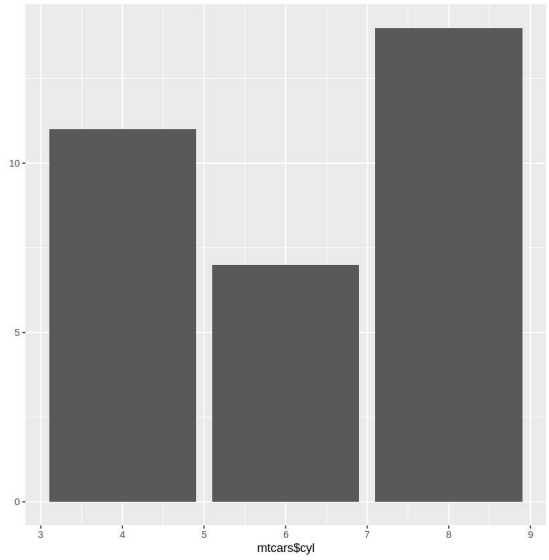
Commençons notre traçage avec des diagrammes à barres. Comme son nom l'indique, il s'agit d'un format de tracé qui affiche vos données à l'aide de barres. Vous en avez probablement déjà vu beaucoup.

Avant de créer un diagramme à barres, importons notre bibliothèque de traçage, `ggplot2`. Il n'est pas nécessaire d'exécuter ce bloc si vous avez déjà exécuté l'import dans "Différences entre bibliothèques R".

```
if("ggplot2" %in% rownames(installed.packages()) == FALSE) {install.packages("ggplot2")}
library(ggplot2)
```

Maintenant que nous avons chargé nos bibliothèques, commençons à tracer. Pour tracer facilement en utilisant `ggplot2`, nous pouvons utiliser la fonction `qplot`, qui a une syntaxe plus simple, comme ceci :

```
qplot(mtcars$cyl, geom = "bar")
```



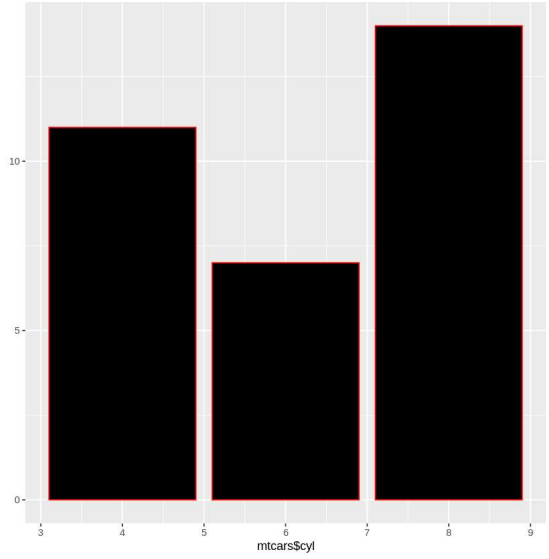
Comme nous pouvons le voir, nous avons tracé un digramme à barres, composé du nombre de chaque élément avec la même valeur. Nous pouvons maintenant exploiter certaines des possibilités de la fonction `qplot` de `ggplot2`.

```
?qplot
```

Vous n'avez pas besoin de comprendre comment fonctionne exactement chaque paramètre de la fonction `qplot`, mais vous pouvez toujours consulter le fichier d'aide pour essayer de trouver ce dont vous avez besoin.

Notre graphique est simple tel qu'il se présente actuellement. Un graphique simple est un excellent choix pour les articles universitaires, mais pour le contenu Internet… le simple fait d'être gris n'attirera pas l'attention des gens. Donnons-lui de la couleur en utilisant les paramètres `color` et `fill`. `color` modifiera la couleur du contour, tandis que `fill` changera la couleur des barres.

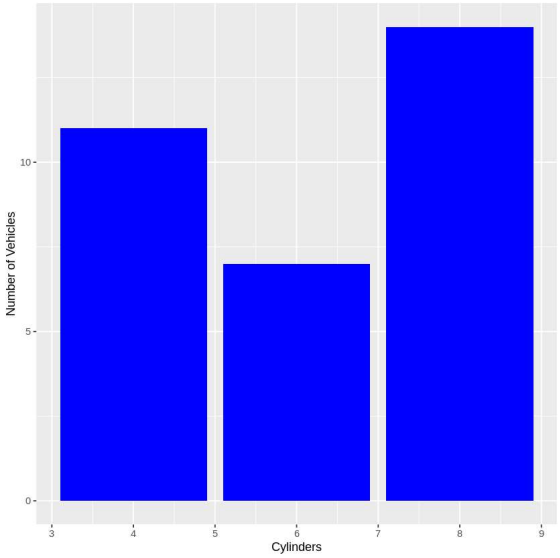
```
qplot(mtcars$cyl, geom = "bar", fill = I("black"), colour = I("red"))
```



Nous pouvons simplement changer le remplissage, car le contour, si aucun paramètre n'est passé, recevra la même valeur que le remplissage. Vous pouvez changer les couleurs en bleu, rose, vert, jaune... Et il y a plus de couleurs ! Voici une liste : http://sape.inf.usi.ch/quick-reference/ggplot2/colour

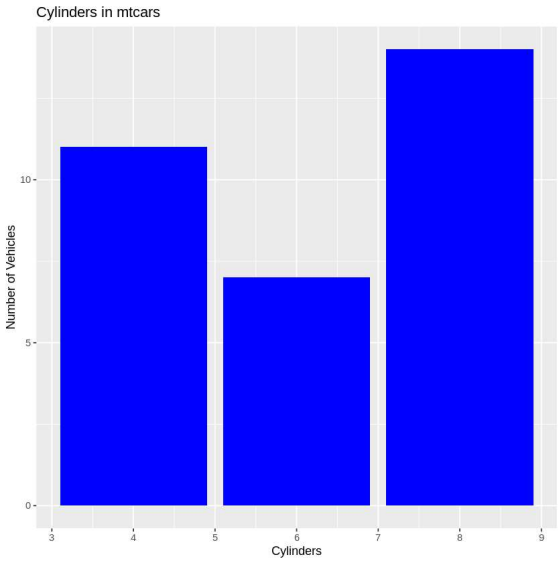
Nous pouvons également changer le nom de nos axes pour rendre le graphique plus compréhensible en passant les paramètres `xlab` et `ylab` (`lab` signifie "label") :

```
qplot(mtcars$cyl, geom = "bar", fill = I("blue"), xlab = "Cylinders", ylab = "Number of Vehicles")
```



Pour terminer notre diagramme à barres, nous pouvons donner un nom à notre graphique. Nous pouvons le faire en utilisant le paramètre `main`.

```
qplot(mtcars$cyl, geom = "bar", fill = I("blue"), xlab = "Cylinders", ylab = "Number of Vehicles", main = "Cylinders in mtcars")
```



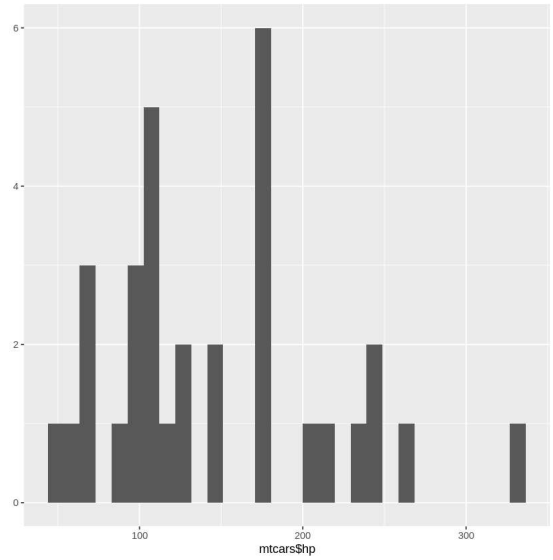
Histogrammes

Les histogrammes peuvent être définis comme une visualisation graphique des décomptes de données. Il s'agit d'un type particulier de diagramme à barres. Un histogramme montre la distribution d'une variable quantitative. Normalement, les diagrammes à barres sont utilisés pour comparer les variables. Habituellement, il n'y a pas d'espace entre les colonnes pour les histogrammes.

Présentons un histogramme simple, en sélectionnant la colonne de puissance dans `mtcars`.

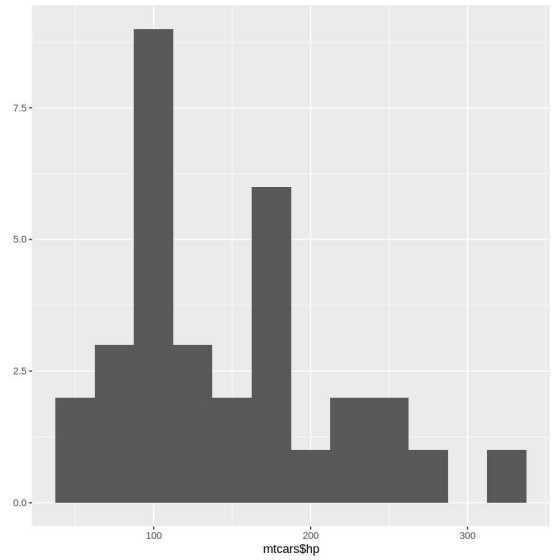
```
qplot(mtcars$hp, geom="histogram")
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



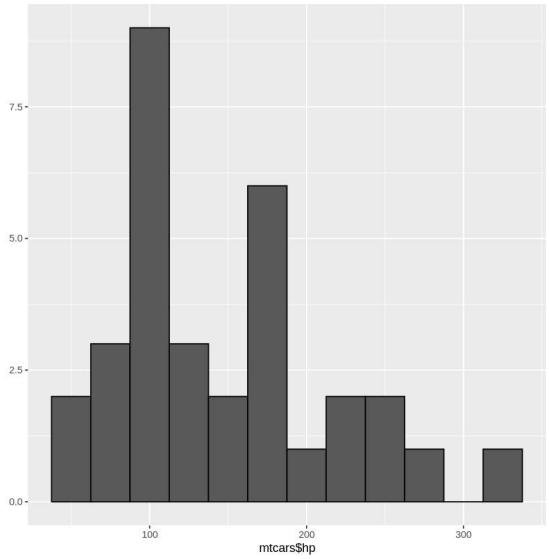
Lorsque vous exécutez le code, vous pouvez recevoir le message d'erreur suivant : `stat_bin()` en utilisant `bins = 30`. Nous pouvons choisir une meilleure valeur avec `binwidth`. Binwidth définit la largeur de votre barre. Améliorons encore notre histogramme, en changeant notre binwidth.

```
qplot(mtcars$hp, geom="histogram", binwidth = 25)
```



Ok, maintenant chaque barre a une largeur de 25 pixels. Cependant, on peut voir que les visualisations de données entre 210 et 260 ch ne sont pas bien illustrées. Encore une fois, ajoutons un contour noir à l'histogramme pour le rendre plus facile à visualiser.

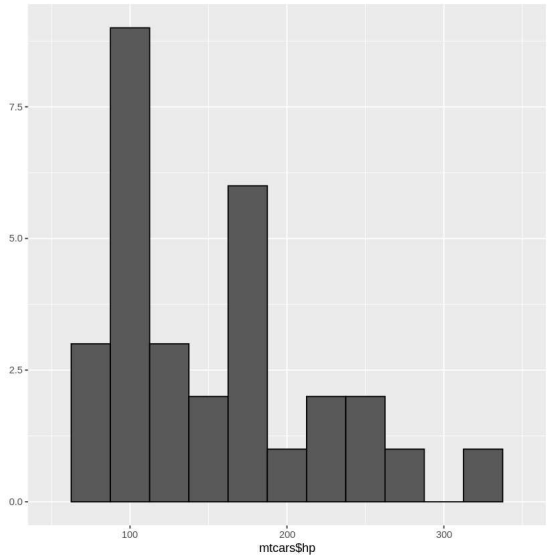
```
qplot(mtcars$hp, geom="histogram", binwidth = 25, colour = I("black"))
```



Nous pouvons également définir des limites pour notre axe x. R ajustera automatiquement les meilleures valeurs selon nos données, donc le résultat ne changera pas lorsque vous utiliserez qplot et une limite adéquate.

```
qplot(mtcars$hp, geom="histogram", binwidth = 25, colour = I("black"),xlim=c(50,350))
```

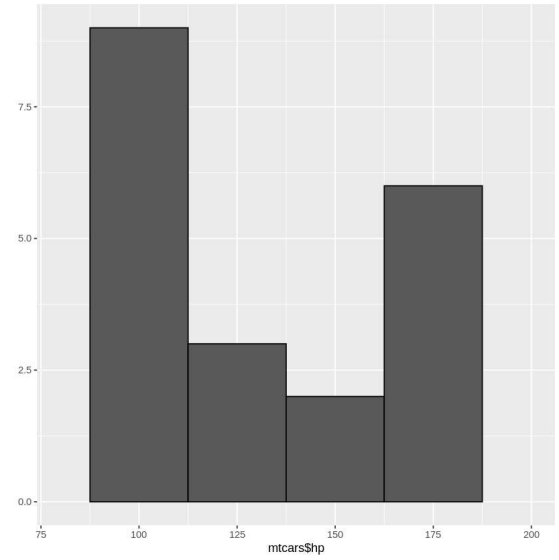
```
Warning message:
“Removed 2 rows containing missing values (`geom_bar()`).”
```



S'il arrive à "couper" une valeur, R affichera un message d'erreur indiquant combien de valeurs ne sont pas affichées. Jouons avec les valeurs xlim, pour voir ce message. Vous pouvez mettre n'importe quelle valeur que vous souhaitez ici.

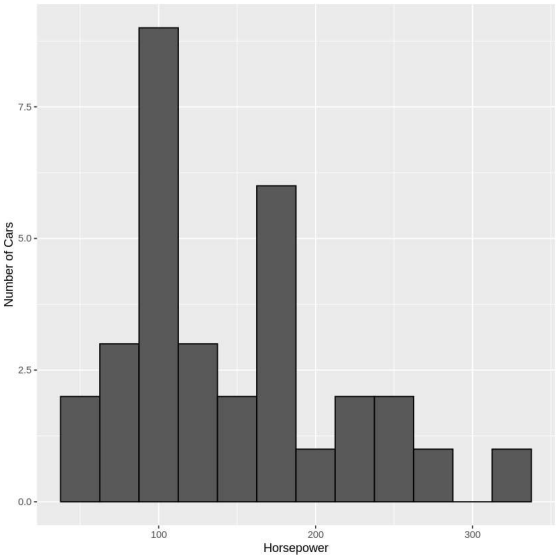
```
qplot(mtcars$hp, geom="histogram", binwidth = 25, colour = I("black"),xlim=c(80,200))
```

```
Warning message:
“Removed 12 rows containing non-finite values (`stat_bin()`).”
Warning message:
“Removed 2 rows containing missing values (`geom_bar()`).”
```



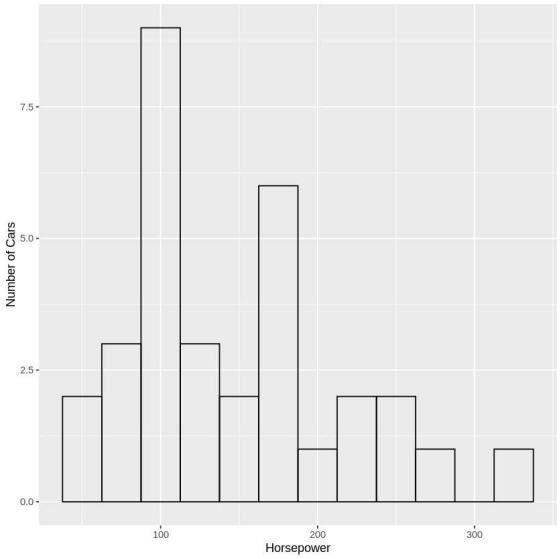
Donnons maintenant un nom à nos axes. Nous utilisons les paramètres `xlab` et `ylab` pour les modifier.

```
qplot(mtcars$hp, geom="histogram", binwidth = 25, colour = I("black"), xlab = "Horsepower", ylab= "Number of Cars")
```



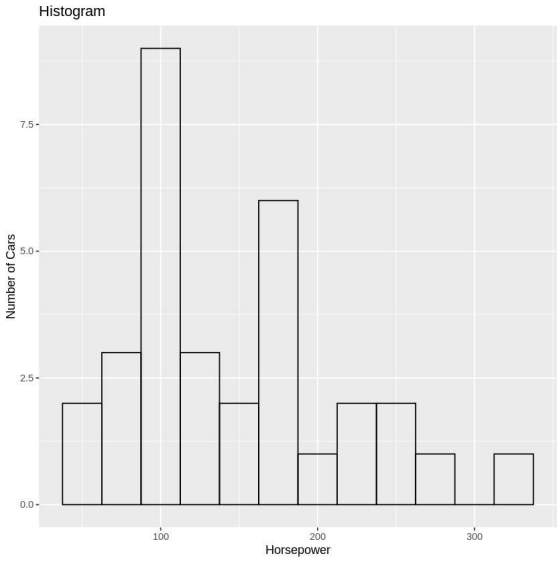
Nous pouvons supprimer la couleur qui remplit nos barres en utilisant le paramètre alpha.

```
qplot(mtcars$hp, geom="histogram", binwidth = 25, colour = I("black"), xlab = "Horsepower", ylab= "Number of Cars", alpha = I(0))
```



Comme nous l'avons fait avec le bar plot, nous pouvons donner un nom à notre graphique. Nous pouvons le faire en utilisant le paramètre `main`.

```
qplot(mtcars$hp, geom="histogram", binwidth = 25, colour = I("black"), xlab = "Horsepower", ylab= "Number of Cars", alpha = I(0),
     main = "Histogram")
```



Conseil d'expert : affinez vos compétences en qplot

Voulez-vous vérifier tous les paramètres de la fonction qplot ? Cliquez ici pour accéder à la documentation de la fonction !

Diagrammes à secteurs

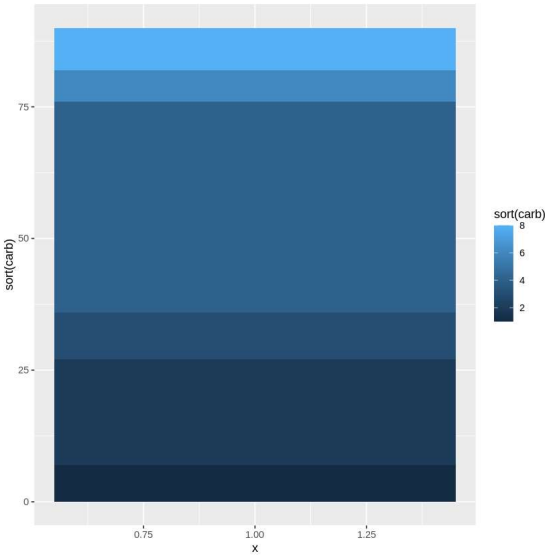
Un diagramme à secteurs est un graphique circulaire, le plus couramment utilisé dans les affaires. Il montre la proportion que chaque partie des données contribue à un total global. Cela se manifeste généralement comme une représentation du nombre de données qualitatives, comme les données démographiques.

Conseil d'expert : choisir le bon type de graphique

Vous ne savez pas quel type de graphique utiliser pour vos données ? Cliquez ici pour en savoir plus !

Un diagramme à secteurs dans ggplot2 est un graphique à barres empilées transformé. Un graphique à barres empilées est un graphique qui empile toutes les valeurs sur l'axe vertical, au lieu de créer des barres séparées pour chaque point de données différent. Commençons par créer un graphique à barres empilées.

```
barp <- ggplot(mtcars, aes(x=1, y=sort(carb), fill=sort(carb))) +
        geom_bar(stat="identity")
print(barp)
```



Nous devons maintenant transformer ce graphique en diagramme à secteurs. Nous allons tracer notre graphique à barres empilées dans le système de coordonnées polaires.

Un très bon rappel sur les coordonnées polaires peut être vu dans le gif suivant :

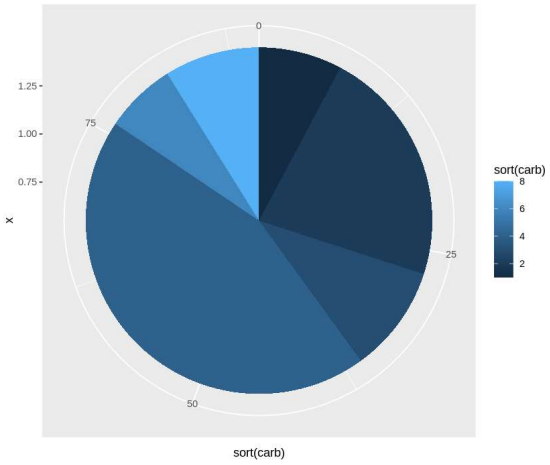
<img src="https://ibm.box.com/shared/static/me1bmbb54gneyr5epweqe88zg8ujkqst.gif"

Source: Wikimedia Commons

Pour ce faire, nous utilisons la fonction d'assistance `coord_polar` :

```
barp <- barp + coord_polar(theta='y')

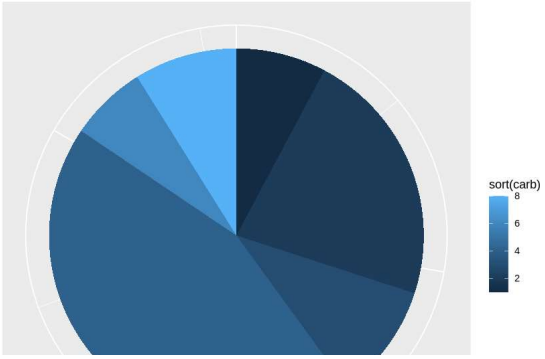
print(barp)
```



Nous pouvons utiliser la fonction d'assistance `theme` afin de ne pas déformer les étiquettes et les axes avec notre transformation :

```
barp <- barp + coord_polar(theta='y')
barp <- barp + theme(
    axis.line=element_blank(),
    axis.text.x=element_blank(),
    axis.text.y=element_blank(),
    axis.ticks=element_blank(),
    axis.title.y=element_blank())
print(barp)
```

```
Coordinate system already present. Adding new coordinate system, which will
replace the existing one.
```



Vous pouvez remarquer que nous avons toujours un contour autour de notre graphique. Nous pouvons étendre la fonction de thème en utilisant le paramètre `panel.background` :

```
barp <- barp + coord_polar(theta='y')
barp <- barp + theme(
    axis.line=element_blank(),
    axis.text.x=element_blank(),
    axis.text.y=element_blank(),
    axis.ticks=element_blank(),
    axis.title.y=element_blank(),
    panel.background=element_blank())
print(barp)
```